Les bases

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Configuration initiale et gestion des fichiers **3**

A. Configuration 3

B. Gestion des états de fichiers 4

C. Exercice : Quiz 7

III. Branches, dépôts distants **7**

A. Le dépôt local et distant 7

B. Les branches 9

C. Exercice : Quiz 10

IV. Essentiel **11**

V. Auto-évaluation **12**

A. Exercice 12

B. Test 12

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : GitBash / Terminal

Prérequis : avoir Git installé sur sa machine

Contexte

Git est un système de contrôle de version largement utilisé dans le domaine du développement logiciel. Il a révolutionné la façon dont les développeurs gèrent et collaborent sur des projets, en offrant un ensemble puissant d'outils et de fonctionnalités.

Depuis sa création en 2005 par Linus Torvalds, le créateur de Linux, Git est devenu le choix prédominant pour le contrôle de version, tant pour les petits projets personnels que pour les grandes entreprises. Ce système de contrôle de version distribué offre aux développeurs une solution efficace pour suivre et gérer les modifications apportées à leur code source au fil du temps. Il permet d'enregistrer les différentes versions d'un projet, de suivre les modifications apportées à chaque fichier et de restaurer ou de revenir à une version antérieure si nécessaire.

Enfin, Git facilite la collaboration entre les développeurs en permettant le partage et la fusion aisés des modifications. Les développeurs peuvent travailler sur leurs propres copies locales du référentiel, effectuer des modifications et les pousser vers un référentiel central ou les partager avec d'autres collaborateurs.

II. Configuration initiale et gestion des fichiers

A. Configuration

Fondamental **Repository (Référentiel)**

Un repository Git, également appelé dépôt Git, ou référentiel, est un endroit où Git stocke l'historique des modifications de votre projet. Il s'agit d'un espace de stockage qui contient toutes les versions de vos fichiers, ainsi que les informations sur les modifications effectuées. On appelle généralement repository un dossier sur une machine contenant du code, et avec Git comme système de gestion de version.

Méthode **Initialiser un repository**

Lorsque vous commencez à mettre en place un projet sur votre machine, vous créez un dossier, et vous ajoutez les fichiers de votre projet à l'intérieur. Si vous voulez utiliser git pour ce projet, il va falloir initialiser le repository dans ce dossier.

Pour commencer, ouvrez le terminal ou l'invite de commande sur votre système d'exploitation. Le but est d'ouvrir le terminal positionné sur le dossier de votre projet. Vous pouvez utiliser la commande « *cd* » pour naviguer vers le répertoire du projet dans lequel vous souhaitez créer le dépôt Git. Par exemple, si votre projet est situé dans le dossier « *mon_projet* », utilisez la commande suivante :

```
cd mon_projet
```

Rien de plus simple que d'initialiser un repository git, il suffit de taper la commande « `git init` ».

```
git init
```

Votre repository, ou dépôt est créé. Cette commande créera un dossier caché nommé « *.git* » qui contiendra toutes les informations de suivi des modifications pour votre projet, vous n'aurez pas à vous en servir.

Vous pouvez maintenant travailler avec votre repository, par exemple en tapant « `git status` », pour afficher les informations sur l'état actuel du dépôt, comme les fichiers non suivis ou modifiés.

```
git status
```

Complément **Git clone**

Lorsque le projet est déjà créé sur un repository distant, nous n'avons pas forcément besoin de passer par `git init` pour initialiser le repository. En effet, si vous utilisez la commande `git clone` pour cloner un dépôt distant, vous n'avez pas besoin d'exécuter git init séparément. La commande git clone effectue à la fois la création du répertoire et l'initialisation du dépôt Git. La commande `git clone` crée automatiquement un nouveau répertoire pour vous, copie tous les fichiers du dépôt distant et initialise un dépôt Git local dans ce répertoire.

Voici comment l'utiliser :

```
git clone <URL_du_dépôt distant>
```

B. Gestion des états de fichiers

Définition **Les zones de stockages**

Dans Git, il existe trois principaux concepts : le workspace (ou répertoire de travail), l'index (ou zone de staging) et le repository (ou dépôt).

- Workspace (répertoire de travail) : le workspace représente le répertoire dans lequel vous travaillez sur vos fichiers. C'est l'espace où vous créez, modifiez et supprimez vos fichiers. Cela correspond à l'état actuel de votre projet sur votre système de fichiers local. Lorsque vous effectuez des modifications dans votre workspace, Git peut détecter ces changements et vous permettre de gérer leur suivi.
- Index (zone de staging) : l'index, également appelé zone de staging ou d'indexation, est un espace intermédiaire entre votre workspace et le dépôt Git. L'index agit comme un espace de préparation pour les modifications que vous souhaitez inclure dans le prochain commit. Lorsque vous effectuez des modifications dans votre workspace, vous utilisez la commande git add pour ajouter ces modifications à l'index. Cela signifie que vous préparez ces modifications pour être incluses dans le prochain commit. L'index agit comme un instantané de l'état que vous souhaitez enregistrer dans le dépôt Git.
- Repository (dépôt) : le repository est l'endroit où Git stocke l'historique complet des modifications de votre projet. Il s'agit d'une base de données contenant tous les commits, les branches, les balises et les métadonnées de votre projet. Le dépôt Git conserve l'intégralité de l'historique de votre projet, y compris tous les commits que vous avez effectués. Vous pouvez considérer le dépôt Git comme la mémoire centrale de votre projet, où toutes les modifications sont enregistrées de manière permanente.

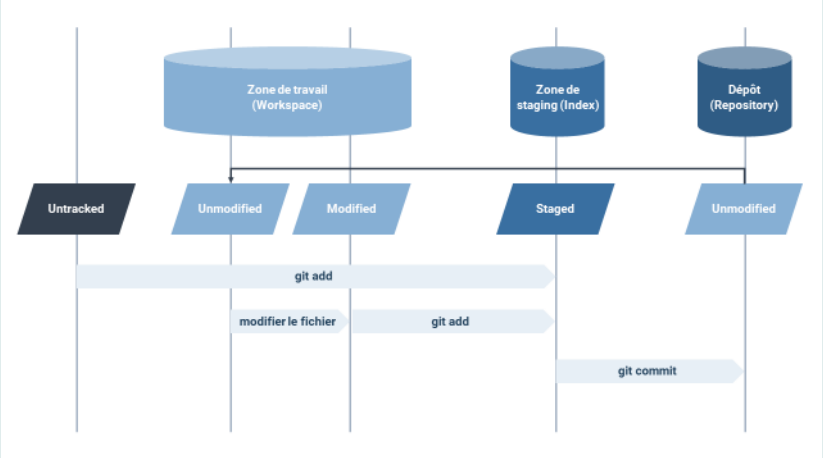
Définition **Les états**

Vous disposez maintenant d'un repository Git fonctionnel. Vous voulez donc faire quelques modifications et valider ces modifications dans votre dépôt chaque fois que votre projet atteint un état que vous souhaitez enregistrer. Dans Git, les fichiers peuvent se trouver dans l'un des quatre états suivants :

- Untracked (non suivi) : Git ne suit pas encore le fichier, ce qui est généralement le cas pour les nouveaux fichiers ajoutés au projet ou ceux qui n'ont pas encore été ajoutés à l'index de Git. Les fichiers non suivis n'apparaissent pas dans les commits et ne font pas partie de l'historique de Git.
- Unmodified (non modifié) : le fichier n'a pas subi de modifications depuis le dernier commit et reste identique à sa version dans l'historique de Git. Les fichiers non modifiés sont importants pour préserver l'intégrité des versions précédentes dans le dépôt Git.
- Modified (modifié) : le fichier a été modifié depuis le dernier commit. Git détecte automatiquement les modifications apportées aux fichiers existants en comparant les versions actuelles avec celles présentes dans l'index ou dans le dernier commit. Les fichiers modifiés peuvent ensuite être ajoutés à l'index pour être inclus dans le prochain commit.

Tous droits réservés à STUDI - Reproduction interdite

- Staged (indexé) : lorsqu'un fichier modifié est ajouté à l'index de Git, il est considéré comme « *staged* » (indexé). Les fichiers indexés sont prêts à être inclus dans le prochain commit. Une fois les fichiers inclus dans un commit, ils passeront à un état différent dans l'historique de Git. Les fichiers indexés sont ceux qui ont été sélectionnés pour être inclus dans le prochain commit.



Indexer un fichier

Lorsque vous modifiez un fichier, Git le détecte automatiquement comme « *modifié* ». Cependant, il ne suit pas automatiquement tous les fichiers modifiés. Vous devez explicitement utiliser la commande `git add` pour ajouter les fichiers modifiés à l'index et les passer à l'état « *indexé* ». Cela signifie que vous les avez préparés pour être inclus dans le prochain commit.

Pour indexer le fichier nom_fichier.html, vous pouvez exécuter cette commande :

```
1 git add nom_fichier.html
```

Si vous voulez ajouter tous les fichiers modifiés, vous pouvez exécuter cette commande :

```
1 git add .
```

Faire un commit

Une fois les fichiers indexés, vous pouvez créer un nouveau commit en utilisant la commande `git commit`. Les fichiers indexés dans ce commit seront enregistrés dans l'historique de Git avec le message du commit correspondant.

Pour faire un commit, vous pouvez exécuter cette commande :

```
1 git commit -m “Message de votre commit”
```

Nous avons utilisé ici l’option -m pour spécifier un message de commit. Vous pouvez aussi en utiliser d’autres pour affiner votre commit. Vous pouvez obtenir la liste complète des options en exécutant `git commit --help` dans votre terminal ou en consultant la documentation officielle de Git.

Voici les options les plus connues :

- `` `-m "message"` `` ou `` `--message "message"` `` : permet de spécifier le message du commit directement en ligne de commande.
- `` `-a` `` ou `` `--all` `` : ajoute automatiquement tous les fichiers modifiés et supprimés à l'index avant de créer le commit. Cela permet d'éviter d'utiliser préalablement git add pour chaque fichier modifié.

Tous droits réservés à STUDI - Reproduction interdite

- `-am "message" `: combinaison des options -a et -m. Cette option permet de créer un commit en incluant automatiquement tous les fichiers modifiés et supprimés, tout en spécifiant le message du commit en ligne de commande.

Vérifier l'état des fichiers

Pour vérifier l'état actuel de vos fichiers, vous pouvez utiliser la commande `git status`. Cette commande vous montrera les fichiers modifiés, les fichiers indexés et les fichiers non suivis, vous permettant de comprendre l'état global de votre dépôt Git.

Complément **Ignorer des fichiers**

Le fichier .gitignore est un outil puissant et essentiel dans Git pour exclure certains fichiers et répertoires de votre dépôt. Lorsque vous travaillez sur un projet, il peut y avoir des fichiers générés automatiquement, des fichiers de configuration spécifiques à votre environnement local ou d'autres fichiers qui ne doivent pas être suivis par Git. Le fichier .gitignore vous permet de spécifier ces fichiers et répertoires à exclure, afin qu'ils ne soient pas ajoutés à l'index et ne fassent pas partie de votre historique de Git. Cela permet de garder votre dépôt propre.

C'est un fichier texte qui contient une liste de règles de filtrage pour exclure certains fichiers et répertoires du suivi de Git. Lorsque Git rencontre un fichier .gitignore, il examine les règles spécifiées à l'intérieur pour déterminer les fichiers à exclure.

Chaque règle est contenue sur une ligne du fichier. Voici un exemple :

```
# Commentaires : les lignes commençant par "#" sont ignorées par Git

# Ignorer les fichiers avec l'extension .log
*.log

# Ignorer le répertoire "build"
/build/

# Ignorer le fichier "config.ini"
config.ini

# Ignorer tous les fichiers et répertoires commençant par "temp_"
temp_*

# Ignorer tous les fichiers dans le répertoire "cache" et ses sous-répertoires
cache/

# Ignorer tous les fichiers portant le nom "backup" dans tous les répertoires
**/backup

# Ignorer tous les fichiers ayant "test" comme préfixe et ".txt" comme extension
test*.txt
```

Remarque **Je ne sais pas quoi mettre dans mon .gitignore**

C'est normal, en fonction de la technologie et de votre environnement, votre fichier .gitignore sera différent. Lors de la création d'un repository, Github vous propose des templates de gitignore. Sinon, vous pouvez aussi utiliser l'outil gitignore.io. C'est un service en ligne pouvant générer des fichiers .gitignore adaptés à votre projet et à votre environnement. Il vous suffit de spécifier les langages de programmation, les frameworks et les environnements que vous utilisez dans votre projet, et il vous génère le fichier gitignore adapté !

Par exemple, si vous travaillez avec Python et Django, vous pouvez saisir « *Python* », puis « *Django* » dans la zone de texte.

Tous droits réservés à STUDI - Reproduction interdite

C. Exercice : Quiz

[solution n°1 p.13]

Question 1

Le command « *git init* » permet de créer un nouveau dépôt Git dans un répertoire existant.

- ○ Vrai
- ○ Faux

Question 2

Quels sont les quatre états des fichiers dans Git ?

- ○ Modified (modifié), Untracked (non suivi), Staged (indexé), Unmodified (non modifié)
- ○ Changed (changé), Untracked (non suivi), Staged (indexé), Unmodified (non modifié)
- ○ Modified (modifié), Ignored (ignoré), Staged (indexé), Unmodified (non modifié)

Question 3

Qu'est-ce que l'état « *Untracked* » dans Git ?

- ○ Cela signifie que le fichier n'a pas été modifié depuis le dernier commit
- ○ Cela signifie que Git ne suit pas encore le fichier
- ○ Cela signifie que le fichier a été ajouté à l'index et est prêt à être inclus dans le prochain commit

Question 4

Quelle commande est utilisée pour ajouter des fichiers modifiés à l'index dans Git ?

- ○ git stage
- ○ git commit
- ○ git add

Question 5

Quelle est la différence entre les états *Modified* (modifié) et *Unmodified* (non modifié) dans Git ?

- ○ *Modified* signifie que le fichier n'a pas été modifié depuis le dernier commit, tandis que *Unmodified* signifie que le fichier a été modifié
- ○ *Modified* signifie que le fichier a été modifié, tandis que *Unmodified* signifie que le fichier n'a pas été modifié depuis le dernier commit
- ○ Il n'y a pas de différence entre les deux états

III. Branches, dépôts distants

A. Le dépôt local et distant

Définition **Dépôt local (Local Repository)**

Le dépôt local est la copie du projet Git présente sur votre machine locale. Il comprend l'ensemble des fichiers du projet, l'historique des commits, les branches et les balises. Lorsque vous initialisez un nouveau dépôt Git avec git init, un dépôt local est créé dans le répertoire du projet. Les commits, les branches et les autres opérations Git

Tous droits réservés à STUDI - Reproduction interdite

sont effectués localement dans le dépôt local. Les modifications apportées aux fichiers et les nouveaux commits sont enregistrés localement avant d'être partagés avec d'autres dépôts. C'est avec ce dépôt que nous travaillons depuis le début du cours.

En effet, au bout d'un moment, vous pourriez ressentir le besoin de sauvegarder votre code sur un serveur, pour pouvoir le partager avec d'autres développeurs, ou pour avoir une sauvegarde fiable. C'est là qu'intervient le dépôt distant.

Définition **Dépôt distant (Remote)**

Le dépôt distant est une copie du dépôt local située sur un serveur ou un service de gestion de code source tel que GitHub, GitLab ou Bitbucket. Il permet la collaboration entre plusieurs développeurs en fournissant un emplacement centralisé pour le partage des modifications.

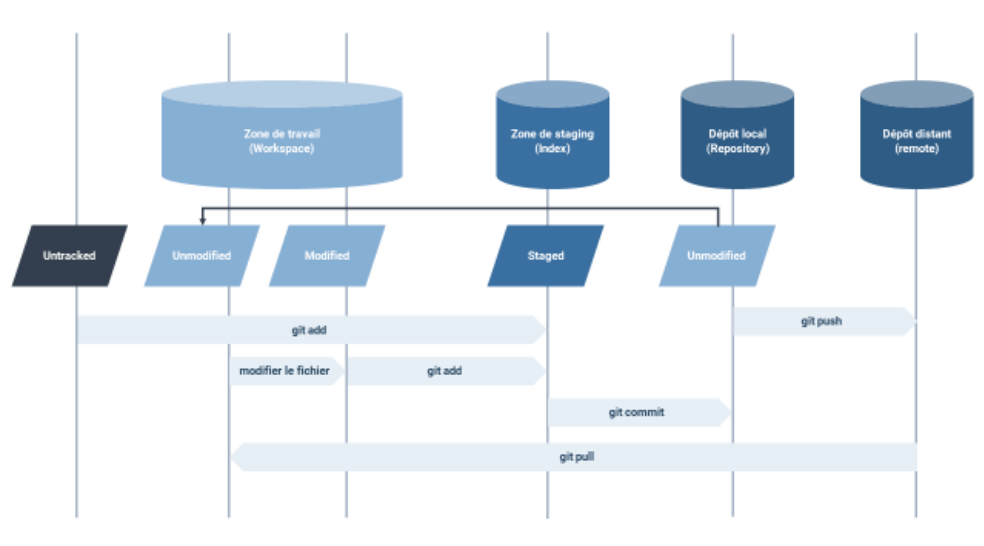
Méthode **Fonctionnement du dépôt distant**

Voici quelques commandes Git couramment utilisées pour travailler avec les remotes :

- `git remote add <nom> <url>` : ajoute une nouvelle remote avec le nom spécifié et l'URL du référentiel distant.
- `git remote -v` : affiche la liste des remotes configurées pour votre dépôt local.
- `git push <remote> <branche>` : pousse les modifications de votre branche locale vers la branche correspondante sur la remote.
- `git pull <remote> <branche>` : récupère les modifications de la branche sur la remote et les fusionne dans votre branche locale.

Vous pouvez cloner le dépôt distant sur votre machine locale à l'aide de la commande `git clone URL_du_dépôt`. Cela crée une copie du dépôt distant dans un nouveau dépôt local.

Les modifications locales effectuées dans le dépôt local peuvent être poussées vers le dépôt distant à l'aide de la commande `git push`. Cela met à jour le dépôt distant avec les derniers commits et modifications. Vous pouvez également récupérer les dernières modifications du dépôt distant vers leur dépôt local en utilisant la commande `git pull`. Cela synchronise le dépôt local avec le dépôt distant en fusionnant les changements. Le dépôt distant sert de point de référence pour la collaboration entre les membres de l'équipe.



Tous droits réservés à STUDI - Reproduction interdite

Attention **Pull avant de push**

Pour pouvoir pousser des modifications sur le serveur, il est important de s'assurer que le repository local est à jour, c'est-à-dire qu'il a bien récupéré les dernières modifications. Si nous essayons de pousser des modifications alors que des modifications sur le serveur ne sont pas présentes en local, git nous générera une erreur.

B. Les branches

Définition **Branche**

Une branche est une ligne de développement indépendante qui permet de travailler sur des modifications de manière isolée par rapport à la branche principale (généralement appelée « *branche principale* » ou « *branche maître* »).

Les branches permettent d'expérimenter de nouvelles fonctionnalités, de corriger des bugs ou de travailler sur des tâches spécifiques sans perturber le code de la branche principale.

Chaque branche possède son propre historique de commits, ce qui facilite la gestion des modifications spécifiques à une fonctionnalité ou à une tâche.

Pourquoi utiliser une branche ?

Quand vous développez, votre application ne fonctionne pas toujours, le temps de développement d'une fonctionnalité peut être assez long.

Imaginez que vous travaillez pour un site de e-commerce, et on vous demande de refondre la gestion des paiements. Vous en avez pour deux mois de développement. Au bout d'un mois de développement, on vous appelle parce qu'il y a un bug critique sur le site, il est inutilisable.

Si vous n'avez pas utilisé de branche, vous n'avez pas de moyen de retrouver le code du site sur lequel vous devez corriger le bug, parce que vous l'avez modifié pour refaire le paiement. Donc vous ne pouvez pas corriger le bug. Pour envoyer une nouvelle version du site, il faudrait finir votre fonctionnalité, ou annuler les modifications que vous avez faites depuis 1 mois. Nous comprenons que c'est impossible.

En utilisant des branches, vous pouvez créer une branche distincte pour travailler sur la nouvelle fonctionnalité, tout en laissant la branche principale intacte. Vous pouvez ensuite travailler sur la fonctionnalité en toute sécurité, en effectuant des commits et des tests sans affecter la version actuellement déployée.

Dans le même temps, un autre membre de votre équipe peut créer une autre branche pour corriger le bug critique dans la version déployée. Cette personne peut effectuer les corrections nécessaires et les tester sans être perturbée par les modifications en cours sur la nouvelle fonctionnalité.

Une fois que la nouvelle fonctionnalité et la correction de bug sont prêtes, les branches peuvent être fusionnées. Cela signifie que les modifications apportées sur les branches de développement seront intégrées à la branche principale, créant ainsi une nouvelle version du logiciel qui comprendra les deux aspects.

Méthode **Création et utilisation des branches**

La création d'une nouvelle branche se fait à l'aide de la commande « `git branch nom_de_branche` ». Par exemple, « `git branch feature-login` » créera une nouvelle branche nommée « *feature-login* ».

Pour basculer vers une branche existante, on utilise la commande « `git checkout nom_de_branche` ». Par exemple, « `git checkout feature-login` » permettra de passer à la branche « *feature-login* ».

Une fois sur une branche, les modifications apportées aux fichiers et les nouveaux commits seront spécifiques à cette branche.

Tous droits réservés à STUDI - Reproduction interdite

Définition **Fusion**

La fusion (ou « *merge* » en anglais) est un concept fondamental dans Git. Il s'agit du processus de fusion de deux branches différentes, généralement dans le but de combiner les modifications apportées sur ces branches et de créer un nouveau commit intégrant ces changements.

Lorsque vous effectuez une fusion, Git analyse les différences entre les deux branches et tente de combiner automatiquement les modifications. Voici les principales étapes d'une fusion sur Git :

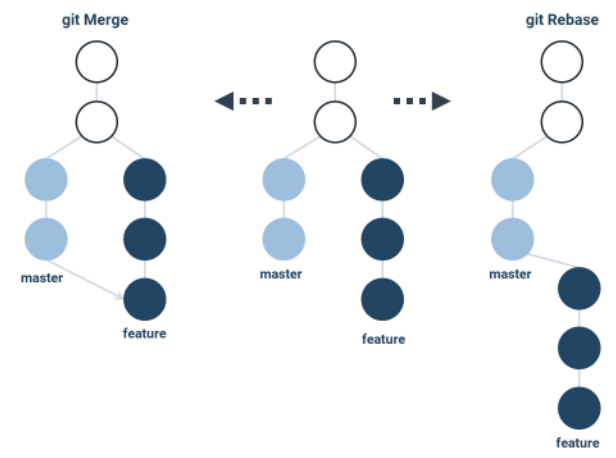
1. Sélection de la branche de destination : vous devez spécifier la branche dans laquelle vous souhaitez fusionner les modifications. Cela peut être la branche principale ou une autre branche de votre choix.
2. Sélection de la branche source : vous devez indiquer la branche à fusionner avec la branche de destination. Cela peut être une branche de développement contenant les modifications que vous souhaitez intégrer.
3. Analyse des différences : Git compare les modifications apportées dans la branche source par rapport à la branche de destination. Il identifie les lignes de code modifiées, ajoutées ou supprimées.
4. Résolution des conflits : si Git détecte des conflits entre les modifications de la branche source et de la branche de destination, il marque ces conflits et demande au développeur de les résoudre manuellement. Les conflits surviennent lorsque Git ne peut pas déterminer automatiquement comment fusionner les modifications en raison de modifications contradictoires sur les mêmes lignes de code.
5. Confirmation de la fusion : une fois les conflits résolus, vous pouvez confirmer la fusion. Git applique alors les modifications de la branche source à la branche de destination, intégrant ainsi les changements.

Lors d'un pull, nous récupérons les modifications distantes. À ce moment-là, Git fait une fusion de la branche distante avec la branche locale, nous pourrions donc rencontrer des conflits, et devoir les résoudre.

Complément **Git rebase**

Git rebase est une commande utilisée pour intégrer les modifications d'une branche sur une autre branche en réécrivant l'historique. Contrairement à la fusion (merge), qui crée un nouveau commit de fusion, le rebase déplace les commits de la branche actuelle au-dessus de la branche cible.

L'avantage du rebase est qu'il produit un historique linéaire et plus propre, car les commits de la branche actuelle semblent avoir été appliqués directement sur la branche cible. Cela facilite la lecture de l'historique et la résolution des conflits.



C. Exercice : Quiz

[solution n°2 p.14]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

En utilisant des branches, il est possible de travailler sur plusieurs fonctionnalités simultanément sans affecter la version principale du code.

- Vrai
- Faux

Question 2

Lors de la fusion d'une branche avec la branche principale, tous les changements apportés à la branche fusionnée sont intégrés dans la branche principale.

- Vrai
- Faux

Question 3

Quel est le principal avantage d'utiliser des branches dans un projet de développement logiciel ?

- Permettre à plusieurs développeurs de travailler simultanément sans se gêner mutuellement.
- Simplifier la gestion des dépôts distants.
- Réduire la taille du code source.

Question 4

Qu'est-ce qu'un dépôt distant dans un système de contrôle de versions, tel que Git ?

- Une branche contenant les modifications finales du code avant sa publication.
- Un emplacement distant où le code source est stocké, accessible par plusieurs développeurs.
- Un outil pour créer des branches locales à partir de la branche principale.

Question 5

Les branches locales d'un projet peuvent être publiées sur un dépôt distant pour permettre la collaboration avec d'autres développeurs.

- Vrai
- Faux

IV. Essentiel

En résumé, Git permet de collaborer sur un même projet tout en évitant les conflits. Initialiser un dépôt Git avec la commande « *git init* » créera un répertoire Git qui permet de suivre les modifications des fichiers qu'il contient. Quant à eux, les dépôts distants (« *remote repository* ») et les dépôts locaux (« *local repository* ») représentent les deux endroits où peuvent être stockés et gérés les fichiers et les versions d'un projet avec Git. Ils permettent de pousser (« *push* ») et de tirer (« *pull* ») les modifications entre le dépôt local et le dépôt distant, favorisant ainsi la collaboration et la synchronisation du code.

Par ailleurs, Git gère les états de fichier et comprend les états de fichier ce qui signifie qu'il suit les modifications apportées aux fichiers d'un projet au fil du temps. Une autre notion importante est celle des branches. Les branches permettent de travailler sur des versions distinctes du code, ce qui facilite le développement parallèle, les tests isolés et les fonctionnalités expérimentales. Il est possible de créer, de basculer sur des branches différentes et fusionner les modifications entre elles.

Tous droits réservés à STUDI - Reproduction interdite

V. Auto-évaluation

A. Exercice

Vous venez d'être embauché dans une entreprise, et vous devez créer votre premier repository.

Question 1

[solution n°3 p.15]

Créez votre repo Git, ajoutez un fichier html, et faites un commit.

Question 2

[solution n°4 p.15]

Créer le repo distant sur Github, et pousser vos fichiers sur Github.

B. Test

Exercice 1 : Quiz

[solution n°5 p.15]

Question 1

Le fichier « *.gitignore* » est utilisé pour ignorer des fichiers ou des répertoires lors de l'indexation avec Git.

- ○ Vrai
- ○ Faux

Question 2

La commande « *git init* » est utilisée pour créer un dépôt Git à partir d'un référentiel distant.

- ○ Vrai
- ○ Faux

Question 3

Qu'est-ce qu'une « *branche* » dans Git ?

- ○ Un fichier contenant des instructions pour ignorer certains fichiers
- ○ Une copie locale d'un dépôt distant
- ○ Une version distincte du code permettant le développement parallèle et les tests isolés

Question 4

Un dépôt distant (*remote repository*) est un référentiel partagé accessible par plusieurs collaborateurs.

- ○ Vrai
- ○ Faux

Question 5

Qu'est-ce qu'un *dépôt local* (*local repository*) dans Git ?

- ○ Une copie du dépôt distant sur la machine locale d'un développeur
- ○ Un fichier de configuration pour spécifier les fichiers à ignorer
- ○ Un environnement de développement pour exécuter du code localement

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 7 Solution n°1

Question 1

Le command « *git init* » permet de créer un nouveau dépôt Git dans un répertoire existant.

- [x] Vrai
- [ ] Faux

La commande « *git init* » est utilisée pour initialiser un nouveau dépôt Git dans un répertoire existant. Cela crée un nouveau sous-répertoire .git qui contient tous les fichiers nécessaires pour le suivi des modifications avec Git.

Question 2

Quels sont les quatre états des fichiers dans Git ?

- [x] Modified (modifié), Untracked (non suivi), Staged (indexé), Unmodified (non modifié)
- [ ] Changed (changé), Untracked (non suivi), Staged (indexé), Unmodified (non modifié)
- [ ] Modified (modifié), Ignored (ignoré), Staged (indexé), Unmodified (non modifié)

Les quatre états des fichiers dans Git sont : Modified (modifié), Untracked (non suivi), Staged (indexé) et Unmodified (non modifié). Chaque état représente l'état actuel d'un fichier dans le cycle de vie de Git.

Question 3

Qu'est-ce que l'état « *Untracked* » dans Git ?

- [ ] Cela signifie que le fichier n'a pas été modifié depuis le dernier commit
- [x] Cela signifie que Git ne suit pas encore le fichier
- [ ] Cela signifie que le fichier a été ajouté à l'index et est prêt à être inclus dans le prochain commit

L'état « Untracked » dans Git signifie que Git ne suit pas encore le fichier. Il s'agit généralement de nouveaux fichiers ajoutés au projet ou de fichiers qui n'ont pas encore été ajoutés à l'index de Git.

Question 4

Quelle commande est utilisée pour ajouter des fichiers modifiés à l'index dans Git ?

- [ ] git stage
- [ ] git commit
- [x] git add

La commande « *git add* » est utilisée pour ajouter des fichiers modifiés à l'index dans Git. Cela permet de préparer les modifications pour le prochain commit en les mettant dans l'état « *Staged* » (indexé).

Question 5

Quelle est la différence entre les états *Modified* (modifié) et *Unmodified* (non modifié) dans Git ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ *Modified* signifie que le fichier n'a pas été modifié depuis le dernier commit, tandis que *Unmodified* signifie que le fichier a été modifié
- ◉ *Modified* signifie que le fichier a été modifié, tandis que *Unmodified* signifie que le fichier n'a pas été modifié depuis le dernier commit
- ○ Il n'y a pas de différence entre les deux états

L'état *Modified* dans Git indique que le fichier a été modifié depuis le dernier commit. En revanche, l'état *Unmodified* indique que le fichier n'a pas été modifié depuis le dernier commit et correspond à la version enregistrée dans l'historique de Git.

Exercice p. 10 **Solution n°2**

Question 1

En utilisant des branches, il est possible de travailler sur plusieurs fonctionnalités simultanément sans affecter la version principale du code.

- ◉ Vrai
- ○ Faux

En utilisant des branches, il est effectivement possible de travailler sur différentes fonctionnalités ou corrections de bugs simultanément sans affecter la version principale du code. Les branches permettent de créer des copies indépendantes du code où chaque développeur peut travailler en isolation.

Question 2

Lors de la fusion d'une branche avec la branche principale, tous les changements apportés à la branche fusionnée sont intégrés dans la branche principale.

- ◉ Vrai
- ○ Faux

Lorsque vous fusionnez une branche avec la branche principale, tous les changements apportés à la branche fusionnée sont effectivement intégrés dans la branche principale. Cela permet de consolider les modifications et de les rendre disponibles dans la version principale du code.

Question 3

Quel est le principal avantage d'utiliser des branches dans un projet de développement logiciel ?

- ◉ Permettre à plusieurs développeurs de travailler simultanément sans se gêner mutuellement.
- ○ Simplifier la gestion des dépôts distants.
- ○ Réduire la taille du code source.

Le principal avantage d'utiliser des branches est de permettre à plusieurs développeurs de travailler simultanément sur des fonctionnalités ou des corrections de bugs différentes sans se gêner mutuellement. Chaque développeur peut travailler dans son propre environnement isolé grâce à une branche distincte.

Question 4

Qu'est-ce qu'un dépôt distant dans un système de contrôle de versions, tel que Git ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une branche contenant les modifications finales du code avant sa publication.
- ◉ Un emplacement distant où le code source est stocké, accessible par plusieurs développeurs.
- ○ Un outil pour créer des branches locales à partir de la branche principale.

Un dépôt distant est un emplacement distant où le code source est stocké et accessible par plusieurs développeurs. Il permet de partager et de synchroniser le code entre les membres de l'équipe de développement. Git est un exemple de système de contrôle de versions qui utilise des dépôts distants.

Question 5

Les branches locales d'un projet peuvent être publiées sur un dépôt distant pour permettre la collaboration avec d'autres développeurs.

- ◉ Vrai
- ○ Faux

Les branches locales d'un projet peuvent effectivement être publiées sur un dépôt distant, ce qui permet la collaboration avec d'autres développeurs. Les développeurs peuvent partager leurs branches avec d'autres membres de l'équipe, qui peuvent ensuite les récupérer et y apporter des modifications avant de les fusionner avec d'autres branches. Cela facilite la collaboration à distance et le travail d'équipe.

p. 12 Solution n°3

Voici les étapes que vous devez suivre :

Dans un premier temps, ouvrez votre terminal ou votre interface de ligne de commande. Ensuite, accédez au répertoire où vous souhaitez créer votre repository Git en utilisant la commande cd. Utilisez la commande suivante pour initialiser un nouveau repository Git :

```
git init
```

À la suite de quoi, créez un fichier HTML nommé « *index.html* » dans le répertoire de votre repository et ajoutez et validez vos modifications en utilisant les commandes suivantes :

```
git add index.html
git commit -m "Ajout du fichier index.html"
```

p. 12 Solution n°4

Voici les étapes que vous devez suivre :

Rendez-vous sur GitHub (https://github.com) et connectez-vous à votre compte. Puis cliquez sur le bouton « *New repository* » pour créer un nouveau repository. Donnez un nom à votre repository et choisissez les options de visibilité appropriées. Une fois le repository créé, vous verrez l'URL du repository distant. Utilisez la commande suivante dans votre terminal pour ajouter le repository distant à votre repository local :

```
git remote add origin <URL_du_repository>
```

Remplacez `<URL_du_repository>` par l'URL du repository GitHub que vous avez créé. Ensuite, utilisez la commande suivante pour pousser vos fichiers vers le repository distant :

```
git push -u origin main
```

Exercice p. 12 Solution n°5

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Le fichier « *.gitignore* » est utilisé pour ignorer des fichiers ou des répertoires lors de l'indexation avec Git.

- [x] Vrai
- [ ] Faux

Le fichier « *.gitignore* » permet d'ignorer certains fichiers ou répertoires lors de l'indexation avec Git. Cela est utile pour exclure des fichiers temporaires, des fichiers de configuration ou des fichiers générés automatiquement, afin de maintenir un référentiel propre et éviter de tracker des fichiers non pertinents.

Question 2

La commande « *git init* » est utilisée pour créer un dépôt Git à partir d'un référentiel distant.

- [ ] Vrai
- [x] Faux

La commande « *git init* » est utilisée pour initialiser un dépôt Git dans un répertoire local vide. Cela crée un référentiel local qui permet de suivre les modifications des fichiers qu'il contient. Pour cloner un référentiel distant et créer un dépôt local à partir de celui-ci, nous utilisons plutôt la commande « *git clone* ».

Question 3

Qu'est-ce qu'une « *branche* » dans Git ?

- [ ] Un fichier contenant des instructions pour ignorer certains fichiers
- [ ] Une copie locale d'un dépôt distant
- [x] Une version distincte du code permettant le développement parallèle et les tests isolés

Une branche dans Git permet de travailler sur une version distincte du code. Cela facilite le développement parallèle, les tests isolés et les fonctionnalités expérimentales. Les branches permettent de travailler sur des modifications sans perturber la branche principale du projet, et elles peuvent être fusionnées ultérieurement pour intégrer les changements.

Question 4

Un dépôt distant (*remote repository*) est un référentiel partagé accessible par plusieurs collaborateurs.

- [x] Vrai
- [ ] Faux

Un dépôt distant (*remote repository*) est un référentiel partagé où plusieurs développeurs peuvent pousser (*push*) et tirer (*pull*) leurs modifications. Cela permet la collaboration entre les membres de l'équipe, le partage du code et la synchronisation des changements effectués par chacun.

Question 5

Qu'est-ce qu'un *dépôt local* (*local repository*) dans Git ?

- [x] Une copie du dépôt distant sur la machine locale d'un développeur
- [ ] Un fichier de configuration pour spécifier les fichiers à ignorer
- [ ] Un environnement de développement pour exécuter du code localement

Un dépôt local (*local repository*) est une copie du dépôt distant située sur la machine locale d'un développeur. Il permet à ce développeur de travailler localement, d'effectuer des modifications, de commiter ses changements et de les partager ultérieurement avec le dépôt distant. C'est dans le dépôt local que sont

Tous droits réservés à STUDI - Reproduction interdite

réalisées la plupart des opérations de développement, avant d'être synchronisées avec le dépôt distant via des commandes de push et de pull. Le dépôt local offre une flexibilité pour expérimenter, développer et valider les changements avant de les partager avec d'autres collaborateurs.

Tous droits réservés à STUDI - Reproduction interdite